

The Trace Formats

What the chain-of-thought tokens are, how each format differs, and how BFS becomes a 1-D array

reference for `src/cot_tokens.py` · `src/cot.py` · 2026-08-24

In one sentence. A chain-of-thought token here is a **symbol in a 42-word alphabet**, written into the **next position** of a one-dimensional sequence — it is not an attention head, not a neuron, and not a learned vector; the attention heads are the **machinery** that each position uses to read the symbols already written, and the whole art of a trace format is arranging the symbols so that the read each next token requires is one the machinery can actually perform.

1 Three different things get called “tokens”

The question “do the CoT tokens belong to attention heads?” is worth answering carefully, because three separate objects are involved and only one of them is a token.

object	what it is	how many
vocabulary symbol	An entry in a fixed alphabet — EXP, SEP, YES, or the node id 7. Just an integer index. <code>CoTVocab</code> assigns them: ids <code>0..max_nodes-1</code> , then ten specials.	42 (<code>max_nodes 32 + 10</code>)
sequence position	A slot in the 1-D array. Each slot holds one symbol, and each slot gets one full pass through the network. This is where the computation happens: more positions = more sequential compute.	up to <code>max_seq_len</code> (192–704)
attention head	A learned read operation, available at every position, in every layer. A head at position t forms a query, matches it against the keys of positions $< t$, and copies back a weighted mixture of their values.	4 per layer $\times$ 2 layers = 8

So: a token does not **belong** to a head. A token is a symbol sitting at a position; a head is a mechanism that position uses to look backwards. Every position has access to all 8 heads, whatever symbol it happens to hold.

The relationship that **does** exist is this one: each trace format is a claim about **what read operation each next token requires**, and a head is what performs that read. When we say the `bfs_expand` parent tokens are “an induction head”, we mean the concrete circuit the model must learn is: attend to the corresponding token one level back and copy it forward. When we say the child tokens are “a lookup”, we mean: attend into the prompt’s edge list at the entries keyed by the adjacent parent id, and copy the partner. Those are the two circuit families the whole project trains, and both are attention patterns — but the **token** is the output symbol, not the head that produced it.

Where the confusion probably comes from. The retired scratchpad CoT (`cot_mode: scratchpad`, `reports/cot-tokens.typ`) used tokens that genuinely **were** parameters: K learnable vectors spliced into the encoder sequence, the same K vectors for every graph. Those were closer to “extra machinery” than to symbols, and they never worked — nothing supervised them, nothing made them graph-dependent, and the model learned to ignore them. The autoregressive CoT

replaced them with **discrete symbols the model emits and re-reads**. That change — from learned vectors to emitted symbols — is the difference between the failed and working halves of this project.

1.1 The model, for reference

CoTTransformer (`src/cot.py`) is a plain decoder-only language model, deliberately unremarkable: token embedding (42×128), optional learned positional embedding, 2 pre-norm blocks of 4-head self-attention + FFN under a **causal** mask, a final LayerNorm, and a tied `lm_head` back to the 42-way vocabulary. About 430k parameters. Nothing in it knows about graphs. Every graph-specific thing lives in the token sequence.

Two implementation details that cost real debugging time: tied embeddings need `std=0.02` init (the default $\mathcal{N}(0,1)$ blows the tied logits up to initial loss ≈ 12.5 and stalls training), and padding needs no key-padding mask because sequences are right-padded under a causal mask — a real query position can only ever see real earlier keys.

2 The prompt: a graph flattened into one dimension

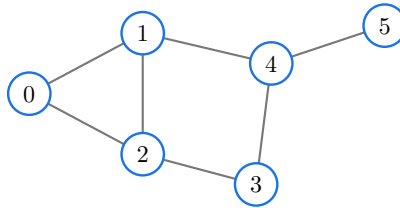
The whole input is one array of integers. Structure:

```

N  v_0 v_1 .. v_{n-1}   E  u_1 w_1  u_2 w_2  ..  u_m w_m   TRACE
└── node roster ─┘      └── edge list, flat pairs ─┘      └─ go

```

Concretely, for the six-node graph used throughout this document:



```

N 0 1 2 3 4 5   E 0 1  0 2  1 2  2 3  3 4  1 4  4 5   TRACE

```

23 tokens. Three things happen during serialization that are not cosmetic:

- **Node ids are randomly permuted per graph.** The generators place components on contiguous id ranges (`connectedness_hard`'s two blobs, `diameter_controlled`'s backbone `0..d`), so literal ids would hand the answer to a model that never traces anything. Permutation also makes memorization expensive, which is half of why circuits win over lookup tables at 32k graphs.
- **Edges are shuffled and normalized** to $(\min, \max)$, so edge order carries nothing.
- **The roster is optional** (`roster: false` drops `N v_0..v_{n-1}`). At fixed n it is informationless distractor mass — a guaranteed non-edge occurrence of every id.

The leak this format created. Connected caterpillars have exactly $n - 1$ edges, disconnected ones $n - 2$ — so the prompt was **two tokens longer** for one class, and the learned positional embedding of `TRACE` read the label straight off sequence length. Decoded accuracy 1.0 by epoch 5, trace exact-match 0.0. This leak is invisible to every GNN baseline, which never sees “edge count” as a feature. Fixed by padding every graph to $n - 1 + k$ edges with diameter-safe chords, $k \sim U\{1, 2, 3\}$, independent of the label.

Lesson. Every model family gets its own leak audit — a representation change creates observables that did not exist before, sequence length first among them.

2.1 Where the loss lives

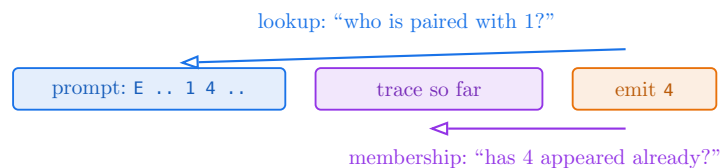
`logits[t]` predicts `tokens[t+1]`, and `_cot_targets` masks every position before `prompt_len` to `-100`. So **the prompt is given, never predicted**; the supervised region is exactly the completion — `trace`, `ANS`, the answer token, `EOS`. The answer-only ablation (`max_trace_len: 0`) keeps this identical architecture and identical prompt distribution and supervises only `ANS YES|NO EOS` — three tokens, one of which carries information. That is the 0.51-at-chance baseline in every headline table.

3 How an algorithm becomes a 1-D array

BFS in ordinary code needs two data structures: a **queue** (the frontier) and a **visited set**. A decoder-only transformer has neither. It has one thing: the sequence of symbols it has already emitted, readable by attention.

Lesson. The trace **is** the data structure. The visited set is “every node id that appears earlier in the trace”. The frontier is “the ids between the previous `SEP` and here”. Attention is the only read primitive. So designing a trace format means choosing an **encoding of the algorithm’s state into emitted symbols** such that every state query the algorithm makes becomes a retrieval the attention pattern can actually express.

That is the whole design problem, and it has a sharp failure mode. If a step of the algorithm is **implied** by the trace rather than **written into** it, that step gets no token, therefore no gradient, therefore no circuit — while every easy neighbouring step is fit perfectly and the loss looks healthy. Four formats below, in the order they were tried, are four points on that lesson.



Every trace token is one or both of these two reads. The formats differ in **how many** of them are given their own supervised token.

4 The five formats, on one graph

All examples below are the real output of `src/cot_tokens.py` on the six-node graph above, BFS from node 0. Levels are `[0] [1 2] [3 4] [5]`; the graph is connected, so the answer is `YES`.

4.1 `bfs_levels` — the compact log

fails

0 SEP 1 2 SEP 3 4 SEP 5 ANS YES EOS (12 tokens)

Frontier by frontier, each level sorted ascending, levels separated by `SEP`. It is the minimal faithful record of the BFS, and it is the format a human would write.

Why it fails. The first token of each level is `min(F)` — the minimum of the entire next frontier, which is the set-union of the neighbourhoods of every node in the current frontier, minus everything seen. That is a **global** computation over the whole prefix, compressed into one all-or-nothing token. Teacher forcing gives it no partial credit: get it wrong and you learn nothing about which parts of the union you had right.

Measured. Teacher-forced answer accuracy 1.0 — the model happily reads `YES/NO` off a gold trace, so the trace does carry the answer — but decoded accuracy at chance and **level-1 accuracy 5.6%**, meaning it cannot even retrieve the neighbours of node 0 with the entire gold prefix handed to it. Decodes are perfectly **shaped** and semantically garbage: ascending levels, plausible sizes, short-trace-before-`NO`, wrong membership. This is Bachmann & Nagarajan’s Clever Hans pitfall, reproduced on graphs.

4.2 bfs_l1 — the atomic lookup, isolated

diagnostic

```
1 2 ANS YES EOS
```

Not a BFS at all — a probe. Emit only the sorted neighbours of node 0, nothing else. It strips away all composition so `trace_em` measures exactly one thing: **can the model do a single content lookup?**

This is the instrument that found the regularization result. With weight decay 0.01 it plateaus at `trace_em` 0.11–0.15; with weight decay 0 it hits 0.95 by epoch 10 and 0.999 by epoch 20, no other change. The later one-knob-at-a-time bisect ran on this probe too: dropout-only 0.93 by epoch 10 (benign), weight-decay-only 0.06/0.09/0.11 (the original plateau, reproduced exactly). It also gave the project its first OOD-positive result — the isolated lookup circuit, trained only on sparse caterpillars, scores `trace_em` 0.55 on dense ER graphs.

4.3 bfs_expand — one line per parent

works on sparse fails on dense

```
0 SEP EXP 0 1 2 SEP EXP 1 4 EXP 2 3 SEP EXP 4 5 EXP 3 SEP EXP 5 ANS YES EOS
```

25 tokens — same information as `bfs_levels`, $\sim 2\times$ the length

Each level is a run of `EXP parent [new children...]` blocks, one block for **every** node of the previous level, empty expansions included. Read it: level 1 expands parent 0 into children 1 2; level 2 expands $1 \rightarrow 4$ and $2 \rightarrow 3$; level 3 expands $4 \rightarrow 5$ and $3 \rightarrow$ nothing; level 4 expands 5 into nothing, which is the explicit frontier-exhausted proof placed right before `ANS`.

The point is the **per-token op**:

- **A parent token after EXP** is a copy of the previous level's children in emission order — an induction head, and nothing more. Note the ordering: the level-3 parents are 4 then 3, not sorted, because that is the order the children were appended. Sorting was deliberately removed; a copy is easier than a copy-and-sort.
- **A child token** is a lookup keyed by the adjacent parent token: find 1 in the prompt's edge list, return its partner. Exactly the `bfs_l1` circuit, which forms at 0.999.

No token requires a minimum over a set. **This is the format behind the headline result:** depth 2, 32k graphs, zero regularization — grokking at epoch 60–70, decoded **0.964 flat across diameters 2 to 18**, and **0.9925** on the adversarial `connectedness_hard_diam`, against 0.51 for the matched answer-only control.

Where it breaks. The children of parent u are `sorted(adj[u] - seen)`. The subtraction is **silent**: every rejected neighbour is a membership test against the entire trace so far that emits **no token**. On sparse caterpillars that is cheap. On `connectedness_hard`'s dense blobs (mean degree ~ 9), nearly every neighbour is already visited, so the hardest operation in the whole algorithm is the one operation with no gradient. Measured: 200 epochs, no grok, decoded flat at ~ 0.52 , and the diagnostic localizes it exactly — parent-copy 0.986, level-1 children 0.998, but **level-2 at 0.272 and level-3 at 0.242**. Format was never the problem (`parse_fail` ~ 0); computation was.

Lesson. Trace locality is not binary — it is parameterized by branching factor. `bfs_expand` made the frontier-min local but left the set-minus implicit, and density is the knob that exposes it.

4.4 bfs_check — every membership test gets a token

works on dense

```
0 SEP EXP 0 1 YES 2 YES
  SEP EXP 1 0 NO 2 NO 4 YES  EXP 2 0 NO 1 NO 3 YES
  SEP EXP 4 1 NO 3 NO 5 YES  EXP 3 2 NO 4 NO
  SEP EXP 5 4 NO
  ANS YES EOS
```

48 tokens — the same BFS again, $\sim 4\times$ `bfs_levels`

Now each parent scans **all** its neighbours in ascending order, and each one is followed by a supervised verdict: YES = new, joins the next frontier; NO = already visited. The two ops are cleanly separated

— the neighbour tokens are a pure prompt lookup (`sorted(adj[u])`), no set arithmetic, the circuit that measures 0.998), and the verdict is a one-bit membership query graded with partial credit. YES/NO are reused from the answer slot, so the vocabulary does not grow and parsing stays unambiguous (the answer is always the token after `ANS`).

Measured — the cleanest causal chain in the project. Identical 32k graphs, identical model and optimizer as the failed `bfs_expand` run; the trace target is the only substantive difference. Loss collapses by epoch ~ 15 , and the full held-out test gives **decoded 0.9972, trace_em 0.9044, flat 0.985–1.000 across diameters 2–8**. The diagnostic before/after is the thesis figure: levels 2–3 go from **0.272 / 0.242** to **0.999 / 0.998**, and the previously silent visited-set test becomes the model’s single most reliable operation — `NO(seen)` at 1.000 over 342,755 positions.

The price is length. `bfs_check` is $\Theta(m)$, not $\Theta(n)$: exactly $1 + L + 2n + 4m$ tokens (L = number of levels). On dense graphs that meant `max_seq_len` 704 and a batch-size cut from 64 to 32 for memory — the one noted confound in that comparison.

4.5 `wl_expand` — the same trick for colour refinement

stalls

The isomorphism chapter’s analog. Input is a graph **pair** as one disjoint union (`G1` at ids `0..n1-1`, `G2` at `n1..n-1`), and the algorithm is 1-WL colour refinement instead of BFS. On a tiny pair of 3-paths:

```
SEP EXP 0 0 0 0 EXP 1 0 0 0 1 EXP 2 0 0 0 EXP 3 0 0 0 EXP 4 0 0 0 1 EXP 5 0 0 0
SEP EXP 0 0 1 0 EXP 1 1 0 0 1 EXP 2 0 1 0 EXP 3 0 1 0 EXP 4 1 0 0 1 EXP 5 0 1 0
SEP SEP 0 0 1 SEP 0 0 1 ANS YES EOS
```

Per round, per node: `EXP u c_old [sorted neighbour colours] c_new`. Colours reuse the node-id token range; the palette is minted in node-id order scanning the union, so colour names are shared across the pair. A fixed number of rounds is emitted for **every** pair regardless of when refinement stabilizes, so trace length depends only on (n, m) and never on the label. The final section makes the multiset comparison explicit — sorted final colours of `G1`, then of `G2` — instead of leaving it as one unsupervised global step.

Measured, and instructive. Teacher-forced ~ 1.0 , loss 0.05, but decoded flat at ~ 0.52 and `trace_em` stalling at ~ 0.003 . The WL-aware diagnostic then **acquitted both predicted culprits** — `c_new` minting 0.989–0.998, the histogram’s first-token minimum 0.999 — and convicted the neighbour-colour gather instead (round 2: 0.888, round 3: 0.909; round 1’s 1.000 is a decoy, since all round-1 colours are 0). The stuck operation is a **two-hop retrieval**: get neighbour ids from the prompt’s edge list, then get each neighbour’s colour from its own record ~ 150 tokens back. The trace made WL’s multiset **hash** local but left the multiset **gathering** silent — the same law at finer grain. Queued fix (`wl_gather`): emit $v\ c(v)$ pairs in neighbour-id order before the sorted list, so every hop is a single supervised lookup.

This is the first case where the diagnostic **rejected the designer’s hypothesis** rather than confirming it. The instrument localizes; it does not merely agree.

5 Side by side

format	length	the op each token demands	outcome
<code>bfs_levels</code>	$n + L$	level-start token = minimum of the whole next frontier . Global, all-or-nothing.	chance
<code>bfs_l1</code>	$\deg(0)$	one prompt lookup, no composition.	0.999
<code>bfs_expand</code>	$3n + L$	parent = copy (induction); child = prompt lookup. Set-minus implicit.	0.96 / 0.9925 0.52 dense
<code>bfs_check</code>	$4m + 2n + L$	neighbour = prompt lookup; verdict = 1-bit membership. Nothing implicit.	0.9972

format	length	the op each token demands	outcome
wl_expand	$\approx R \cdot (3n + 2m)$	c _{old} = copy; c _{new} = associative match. Gather is two-hop.	~ 0.52

The column that predicts the outcome is the third one, not the second. Length is a cost; implicitness is the failure.

6 The law these five formats trace out

Lesson. The silent-op law (four confirmed instances). If a step of the algorithm is implied by the trace rather than written into it, that step receives no gradient and no circuit forms — while every neighbouring easy step is fit perfectly, the loss falls, and teacher-forced accuracy reaches 1.0. A healthy-looking loss curve is compatible with the hardest operation being completely unlearned.

The four instances, in order: the frontier-minimum (**bfs_levels**), the visited-set subtraction under density (**bfs_expand** on **connectedness_hard**), the same subtraction made explicit and thereby fixed (**bfs_check** — the first **constructive** instance, on a task that had already failed), and the neighbour-colour gather (**wl_expand**).

Two axes appear to govern it, and making them measurable is the open question:

1. **Retrieval distance** — how far back the evidence for the next token lives. The WL gather reaches ~ 150 tokens back; a **bfs_expand** child reaches into the prompt, which is close and highly patterned.
2. **Aggregation arity** — how many prefix tokens must be combined into one emitted token. A frontier-minimum aggregates the entire frontier’s neighbourhoods; a **bfs_check** verdict aggregates one bit; a dense-blob visited-set test aggregates the whole trace so far.

Everything that worked sits at low arity with short, patterned retrieval. Everything that failed violates one of the two. Whether a statistic over these axes **predicts** trainability before training — rather than explaining it afterwards — is the interesting version of the question, and the five formats above are already five labelled data points for it.

7 Practical notes

Decoding. Greedy, with a per-layer KV cache (**generate()**), verified token-identical to the naive re-forward loop it replaced — 3.5× on MPS at **max_new=120**, more at longer traces. Rows advance at their own cursor; cache slots past a row’s cursor are masked until that row writes them.

Metrics. **answer_acc** (decoded YES/NO correct — the primary number), **trace_em** (the decoded trace exactly matches the canonical one), **parse_fail** (no well-formed **ANS YES|NO** at all — a **format** failure, distinct from a computational one), and the teacher-forced answer accuracy logged every epoch as a cheap proxy. Note that the teacher-forced number conditions on the **gold** trace and is therefore an upper bound; the entire **bfs_levels** disaster hid behind a teacher-forced 1.0.

Adding a format. Write a `..._trace(n, edges, start, vocab) -> (tokens, answer)` function in `src/cot_tokens.py`, add a branch in `build_cot_sequences`, and set `trace_format` in the config. The answer is asserted against the dataset label at build time, which is how a mislabelled generator gets caught immediately rather than after a training run. Check the measured length against `max_seq_len` on the **full** dataset, not a sample — for **bfs_check** the 4k-sample maximum was 620 and the true maximum was 665.

Source: `src/cot_tokens.py`, `src/cot.py`, `src/train.py`. Narrative: `docs/CHANGELOG.md` 2026-07-03 → 07-08, `reports/cot-journey.typ`. Every example in this document is real output of the code, not hand-written.